

Módulo 3B — Excepciones y Fail Fast

Implementación completa: DomainException + 7 excepciones + integración en Value Objects

CommerX · Arquitectura de Software Profesional · .NET 9 · Clean Architecture · DDD

Objetivo: Implementar la clase base `DomainException`, las 7 excepciones personalizadas del módulo Customer y actualizar cada Value Object para lanzar su excepción específica, reemplazando definitivamente los `ArgumentException` del Módulo 2C.

9 · Ejemplo práctico — Código completo comentado línea por línea

Estructura de carpetas a crear

```
src/
+--- Domain/
|   +--- Common/
|   |   +--- Exceptions/
|   |   |   +--- DomainException.cs          <-- NUEVO (base abstracta)
|   |   +--- Entities/
|   |   +--- ValueObjects/
|   +--- Customers/
|   |   +--- Entities/
|   |   |   +--- Customer.cs
|   |   +--- Exceptions/                      <-- NUEVA carpeta
|   |   |   +--- InvalidFullNameException.cs  <-- NUEVO
|   |   |   +--- InvalidLastNameException.cs  <-- NUEVO
|   |   |   +--- InvalidDocumentException.cs  <-- NUEVO
|   |   |   +--- InvalidEmailException.cs     <-- NUEVO
|   |   |   +--- InvalidPhoneException.cs     <-- NUEVO
|   |   |   +--- InvalidAddressException.cs   <-- NUEVO
|   |   |   +--- InvalidAgeException.cs       <-- NUEVO
|   |   +--- ValueObjects/
|   |   |   +--- FullName.cs                  <-- MODIFICADO
|   |   |   +--- LastName.cs                  <-- MODIFICADO
|   |   |   +--- Document.cs                  <-- MODIFICADO
|   |   |   +--- EmailAddress.cs              <-- MODIFICADO
|   |   |   +--- Phone.cs                     <-- MODIFICADO
|   |   |   +--- Address.cs                   <-- MODIFICADO
|   |   |   +--- BirthDate.cs                 <-- MODIFICADO
```

Paso 1 de 9 — Clase base abstracta DomainException

Archivo: `src/Domain/Common/Exceptions/DomainException.cs`

Toda excepción del dominio hereda de esta clase. Permite capturar cualquier error de dominio con un solo `catch (DomainException)`.

```
1 // Espacio de nombres: capa Domain, carpeta Common/Exceptions
2 namespace CommerX.Domain.Common.Exceptions;
3
4 // abstract: no se puede instanciar directamente
5 // Sirve como tipo base para capturar cualquier error de dominio
6 public abstract class DomainException : Exception
7 {
8     // protected: solo accesible desde clases hijas
9     // Recibe el mensaje y lo delega a Exception de .NET
10    protected DomainException(string message) : base(message) { }
11 }
```

Paso 2 de 9 — InvalidFullNameException

Archivo: `src/Domain/Customers/Exceptions/InvalidFullNameException.cs`

```
1 using CommerX.Domain.Common.Exceptions;
2
3 namespace CommerX.Domain.Customers.Exceptions;
4
5 // sealed: excepción final, no puede heredarse
6 public sealed class InvalidFullNameException : DomainException
7 {
8     // Recibe el valor inválido para incluirlo en el mensaje de error
9     public InvalidFullNameException(string value)
10         : base($"El nombre '{value}' no es válido. Debe tener entre 2 y 100 caracteres.") { }
11 }
```

Paso 3 de 9 — InvalidLastNameException

Archivo: `src/Domain/Customers/Exceptions/InvalidLastNameException.cs`

```
1 using CommerX.Domain.Common.Exceptions;
2
3 namespace CommerX.Domain.Customers.Exceptions;
4
5 public sealed class InvalidLastNameException : DomainException
6 {
7     public InvalidLastNameException(string value)
8         : base($"El apellido '{value}' no es válido. Debe tener entre 2 y 100 caracteres.") { }
9 }
```

Paso 4 de 9 — InvalidDocumentException

Archivo: `src/Domain/Customers/Exceptions/InvalidDocumentException.cs`

```
1 using CommerX.Domain.Common.Exceptions;
2
3 namespace CommerX.Domain.Customers.Exceptions;
4
5 public sealed class InvalidDocumentException : DomainException
6 {
7     // Documento: solo dígitos, entre 7 y 15 caracteres
8     public InvalidDocumentException(string value)
9         : base($"El documento '{value}' no es válido. Solo dígitos, entre 7 y 15 caracteres.")
10     { }
11 }
```

Paso 5 de 9 — InvalidEmailException

Archivo: `src/Domain/Customers/Exceptions/InvalidEmailException.cs`

```
1 using CommerX.Domain.Common.Exceptions;
2
3 namespace CommerX.Domain.Customers.Exceptions;
4
5 public sealed class InvalidEmailException : DomainException
6 {
7     // Incluye el valor exacto para facilitar diagnóstico en logs
8     public InvalidEmailException(string value)
9         : base($"El email '{value}' no tiene formato válido (ej: usuario@dominio.com).") { }
10 }
```

Paso 6 de 9 — InvalidPhoneException

Archivo: `src/Domain/Customers/Exceptions/InvalidPhoneException.cs`

```
1 using CommerX.Domain.Common.Exceptions;
2
3 namespace CommerX.Domain.Customers.Exceptions;
4
5 public sealed class InvalidPhoneException : DomainException
6 {
7     public InvalidPhoneException(string value)
8         : base($"El teléfono '{value}' no es válido. Solo dígitos, entre 7 y 15 caracteres.")
9     { }
10 }
```

Paso 7 de 9 — InvalidAddressException

Archivo: `src/Domain/Customers/Exceptions/InvalidAddressException.cs`

```
1 using CommerX.Domain.Common.Exceptions;
2
3 namespace CommerX.Domain.Customers.Exceptions;
4
5 public sealed class InvalidAddressException : DomainException
6 {
7     public InvalidAddressException(string value)
8         : base($"La dirección '{value}' no es válida. Debe tener entre 5 y 200 caracteres.") {
9     }
10 }
```

Paso 8 de 9 — InvalidAgeException

Archivo: `src/Domain/Customers/Exceptions/InvalidAgeException.cs`

```
1 using CommerX.Domain.Common.Exceptions;
2
3 namespace CommerX.Domain.Customers.Exceptions;
4
5 public sealed class InvalidAgeException : DomainException
6 {
7     // Recibe DateOnly para mostrar la fecha exacta que causó el error
8     public InvalidAgeException(DateOnly birthDate)
9         : base($"La fecha de nacimiento '{birthDate:dd/MM/yyyy}' no cumple el requisito de
10 mayoría de edad (18 años).") { }
11 }
```

Paso 9 de 9 — Value Objects actualizados

Se reemplaza cada `ArgumentException` del Módulo 2C por su excepción personalizada correspondiente. El patrón es idéntico en todos: mismo constructor privado, mismo `Create()`, solo cambia el tipo lanzado.

FullName.cs — actualizado

Archivo: `src/Domain/Customers/ValueObjects/FullName.cs`

```
1 using CommerX.Domain.Common.ValueObjects;
2 using CommerX.Domain.Customers.Exceptions;
3
4 namespace CommerX.Domain.Customers.ValueObjects;
5
6 // sealed record: immutable, igualdad por valor, no heredable
7 public sealed record FullName : ValueObject
8 {
9     // Propiedad de solo lectura: inmutabilidad total
10    public string Value { get; }
11
12    // Constructor privado: solo Create() puede instanciar
13    private FullName(string value) => Value = value;
14
15    // Método factory: validación + normalización + construcción
16    public static FullName Create(string value)
17    {
18        // Fail Fast: vacío o nulo → excepción inmediata
19        if (string.IsNullOrWhiteSpace(value))
20            throw new InvalidFullNameException(value ?? string.Empty);
21
22        // Normalización: eliminar espacios extremos
23        var trimmed = value.Trim();
24
25        // Fail Fast: longitud fuera de rango → excepción de dominio
26        if (trimmed.Length < 2 || trimmed.Length > 100)
27            throw new InvalidFullNameException(trimmed);
28
29        // Solo llega aquí si el valor es completamente válido
30        return new FullName(trimmed);
31    }
32 }
```

LastName.cs — actualizado

Archivo: `src/Domain/Customers/ValueObjects/LastName.cs`

```
1 using CommerX.Domain.Common.ValueObjects;
2 using CommerX.Domain.Customers.Exceptions;
3
```

```
4 namespace CommerX.Domain.Customers.ValueObjects;
5
6 public sealed record LastName : ValueObject
7 {
8     public string Value { get; }
9     private LastName(string value) => Value = value;
10
11     public static LastName Create(string value)
12     {
13         if (string.IsNullOrEmpty(value))
14             throw new InvalidLastNameException(value ?? string.Empty);
15
16         var trimmed = value.Trim();
17
18         if (trimmed.Length < 2 || trimmed.Length > 100)
19             throw new InvalidLastNameException(trimmed);
20
21         return new LastName(trimmed);
22     }
23 }
```

Document.cs — actualizado

Archivo: `src/Domain/Customers/ValueObjects/Document.cs`

```
1 using System.Text.RegularExpressions;
2 using CommerX.Domain.Common.ValueObjects;
3 using CommerX.Domain.Customers.Exceptions;
4
5 namespace CommerX.Domain.Customers.ValueObjects;
6
7 public sealed record Document : ValueObject
8 {
9     public string Value { get; }
10    private Document(string value) => Value = value;
11
12    public static Document Create(string value)
13    {
14        // Fail Fast: vacío
15        if (string.IsNullOrEmpty(value))
16            throw new InvalidDocumentException(value ?? string.Empty);
17
18        var trimmed = value.Trim();
19
20        // Regex: solo dígitos, longitud entre 7 y 15
21        if (!Regex.IsMatch(trimmed, @"^\d{7,15}$"))
22            throw new InvalidDocumentException(trimmed);
23
24        return new Document(trimmed);
25    }
26 }
```

EmailAddress.cs — actualizadoArchivo: `src/Domain/Customers/ValueObjects/EmailAddress.cs`

```
1 using System.Text.RegularExpressions;
2 using CommerX.Domain.Common.ValueObjects;
3 using CommerX.Domain.Customers.Exceptions;
4
5 namespace CommerX.Domain.Customers.ValueObjects;
6
7 // Value Object que encapsula la validación de formato de email
8 public sealed record EmailAddress : ValueObject
9 {
10     // Regex compilada: mejor rendimiento en llamadas repetidas
11     private static readonly Regex EmailRegex =
12         new(@"^[^@\s]+@[^@\s]+\.[^@\s]+$", RegexOptions.Compiled);
13
14     public string Value { get; }
15     private EmailAddress(string value) => Value = value;
16
17     public static EmailAddress Create(string value)
18     {
19         // Fail Fast 1: nulo o vacío
20         if (string.IsNullOrEmpty(value))
21             throw new InvalidEmailException(value ?? string.Empty);
22
23         // Normalización: sin espacios, todo en minúsculas
24         var normalized = value.Trim().ToLowerInvariant();
25
26         // Fail Fast 2: formato inválido → excepción de dominio específica
27         if (!EmailRegex.IsMatch(normalized))
28             throw new InvalidEmailException(normalized);
29
30         // Valor válido y normalizado: se construye el VO
31         return new EmailAddress(normalized);
32     }
33 }
```

Phone.cs — actualizadoArchivo: `src/Domain/Customers/ValueObjects/Phone.cs`

```
1 using System.Text.RegularExpressions;
2 using CommerX.Domain.Common.ValueObjects;
3 using CommerX.Domain.Customers.Exceptions;
4
5 namespace CommerX.Domain.Customers.ValueObjects;
6
7 public sealed record Phone : ValueObject
8 {
9     public string Value { get; }
10     private Phone(string value) => Value = value;
11 }
```

```
12     public static Phone Create(string value)
13     {
14         if (string.IsNullOrEmpty(value))
15             throw new InvalidPhoneException(value ?? string.Empty);
16
17         var trimmed = value.Trim();
18
19         // Solo dígitos, entre 7 y 15 caracteres
20         if (!Regex.IsMatch(trimmed, @"^\d{7,15}$"))
21             throw new InvalidPhoneException(trimmed);
22
23         return new Phone(trimmed);
24     }
25 }
```

Address.cs — actualizado

Archivo: `src/Domain/Customers/ValueObjects/Address.cs`

```
1     using CommerX.Domain.Common.ValueObjects;
2     using CommerX.Domain.Customers.Exceptions;
3
4     namespace CommerX.Domain.Customers.ValueObjects;
5
6     public sealed record Address : ValueObject
7     {
8         public string Value { get; }
9         private Address(string value) => Value = value;
10
11         public static Address Create(string value)
12         {
13             if (string.IsNullOrEmpty(value))
14                 throw new InvalidAddressException(value ?? string.Empty);
15
16             var trimmed = value.Trim();
17
18             // Longitud mínima 5 (ej: "Av. B") y máxima 200
19             if (trimmed.Length < 5 || trimmed.Length > 200)
20                 throw new InvalidAddressException(trimmed);
21
22             return new Address(trimmed);
23         }
24     }
```

BirthDate.cs — actualizado (regla de mayoría de edad)

Archivo: `src/Domain/Customers/ValueObjects/BirthDate.cs`

```
1     using CommerX.Domain.Common.ValueObjects;
2     using CommerX.Domain.Customers.Exceptions;
3
4     namespace CommerX.Domain.Customers.ValueObjects;
5
```




```
6 // VO que encapsula la regla de mayoría de edad del dominio – CU-CLI-001
7 public sealed record BirthDate : ValueObject
8 {
9     // Constante: la regla de negocio está nombrada, no es un número mágico
10    private const int MinimumAge = 18;
11
12    public DateOnly Value { get; }
13    private BirthDate(DateOnly value) => Value = value;
14
15    public static BirthDate Create(DateOnly value)
16    {
17        // Fecha de hoy como referencia para calcular la edad
18        var today = DateOnly.FromDateTime(DateTime.Today);
19
20        // Fail Fast: fecha futura es inválida como fecha de nacimiento
21        if (value > today)
22            throw new InvalidAgeException(value);
23
24        // Cálculo base: diferencia de años
25        var age = today.Year - value.Year;
26
27        // Ajuste preciso: si el cumpleaños de este año aún no ocurrió
28        if (value.AddYears(age) > today) age--;
29
30        // Fail Fast: regla de mayoría de edad – CU-CLI-001
31        if (age < MinimumAge)
32            throw new InvalidAgeException(value);
33
34        return new BirthDate(value);
35    }
36 }
```

Vista previa — Cómo Application captura las excepciones de dominio

Este patrón se desarrolla en profundidad en el Módulo 6. Se incluye aquí para visualizar el circuito completo desde el dominio hasta la capa superior.

```
1 // Archivo: src/Application/UseCases/CreateCustomerUseCase.cs (preview Módulo 6)
2 using CommerX.Domain.Common.Exceptions;
3 using CommerX.Domain.Customers.Exceptions;
4 using CommerX.Domain.Customers.ValueObjects;
5
6 namespace CommerX.Application.UseCases;
7
8 public class CreateCustomerUseCase
9 {
10     public void Execute(string email, string document, DateOnly birthDate)
11     {
12         try
13         {
14             // El dominio valida internamente y lanza si algo falla
15             var emailVO = EmailAddress.Create(email);
16             var docVO = Document.Create(document);
17             var birthVO = BirthDate.Create(birthDate);
18         }
19         // Captura específica: saber exactamente qué campo falló
20         catch (InvalidEmailException ex)
21         {
22             // Application traduce: excepción de dominio → respuesta de aplicación
23             throw new ValidationException("email", ex.Message);
24         }
25         // Captura general: cualquier otra violación de dominio no identificada
26         catch (DomainException ex)
27         {
28             // Posible gracias a la herencia: DomainException es la base de todas
29             throw new ValidationException("campo", ex.Message);
30         }
31     }
32 }
```

10 · Actividades prácticas — CU-CLI-001 Alta de Cliente

Actividad 1 – Crear la jerarquía base

Dificultad: Básica | Tiempo estimado: 15 min

Instrucciones:

- 1. Crea la carpeta `src/Domain/Common/Exceptions/` en tu proyecto
- 2. Crea el archivo `DomainException.cs` con la clase abstracta base
- 3. Crea la carpeta `src/Domain/Customers/Exceptions/`
- 4. Crea los 7 archivos de excepciones con sus constructores y mensajes
- 5. Verifica que todos los `namespace` sean correctos
- 6. Compila el proyecto Domain – no debe haber errores

Pregunta de reflexion: ¿Por qué `DomainException` es `abstract` y no puede instanciarse directamente? ¿Qué problemas evita ese diseño?

Actividad 2 – Actualizar los 7 Value Objects

Dificultad: Intermedia | Tiempo estimado: 30 min

Instrucciones:

- 1. Abre cada uno de los 7 VOs creados en el Módulo 2C
- 2. Reemplaza cada `throw new ArgumentException(...)` por su excepción específica
- 3. Agrega el `using` de `CommerX.Domain.Customers.Exceptions` en cada archivo
- 4. Para `BirthDate`, asegúrate de pasar el `DateOnly` al constructor de `InvalidAgeException`
- 5. Compila la solución completa – sin referencias rotas

Pregunta de reflexion: ¿Qué ventaja tiene `InvalidDocumentException` sobre `ArgumentException` cuando el sistema crece a 50 VOs diferentes?

Actividad 3 – Verificar Fail Fast con pruebas manuales

Dificultad: Intermedia | Tiempo estimado: 20 min

Instrucciones:

- 1. En `Program.cs`, escribe: `EmailAddress.Create("noesvalido")` dentro de un `try/catch`
- 2. Captura `InvalidEmailException` e imprime `ex.Message` en consola
- 3. Verifica que el mensaje incluye el valor `"noesvalido"`
- 4. Repite con `BirthDate.Create(DateOnly.FromDateTime(DateTime.Today.AddYears(-10)))`
- 5. Comprueba que `InvalidAgeException` incluye la fecha exacta en el mensaje
- 6. Prueba también capturando `DomainException` en lugar del tipo específico

Pregunta de reflexion: Al capturar `DomainException` en vez de `InvalidEmailException`, ¿pierdes informacion util? ¿En que capa es aceptable usar la captura generica?

Actividad 4 – Analisis de arquitectura

Dificultad: Reflexiva | Tiempo estimado: 15 min

Escenario: Un compañero propone: "Para no duplicar código, pondre todas las excepciones en una sola clase `DomainException` con un enum que indique el tipo de error."

Preguntas:

- 1. ¿Que principio SOLID viola esa propuesta?
- 2. ¿Como impacta en la capacidad de capturar errores específicos en Application?
- 3. ¿Que desventaja tendría al agregar un nuevo VO al sistema?
- 4. Escribe un argumento técnico de dos párrafos para defender la jerarquía actual.

Actividad 5 – Desafío: nueva regla de negocio

Dificultad: Avanzada | Tiempo estimado: 25 min

Nueva regla: El equipo decide aceptar solo documentos que comiencen con los prefijos `20`, `23` o `27` (CUIL/CUIT argentino).

Instrucciones:

- 1. Modifica `Document.cs` para agregar esta validación de prefijo
- 2. La excepción lanzada sigue siendo `InvalidDocumentException`
- 3. El mensaje debe indicar exactamente que prefijos son válidos
- 4. Prueba con `Document.Create("30123456")` – debe lanzar excepción
- 5. Prueba con `Document.Create("20123456")` – debe pasar
- 6. Prueba con `Document.Create("23456789")` – debe pasar

Pregunta de reflexión: ¿En cuántas clases fue necesario hacer cambios? ¿Que principio de Clean Architecture hizo posible que el cambio estuviera contenido en un solo lugar?

Resumen del Módulo 3

Archivos nuevos	<code>DomainException.cs</code> (base abstracta) y las 7 excepciones en <code>Domain/Customers/Exceptions/</code>: <code>InvalidFullNameException</code>, <code>InvalidLastNameException</code>, <code>InvalidDocumentException</code>, <code>InvalidEmailException</code>, <code>InvalidPhoneException</code>, <code>InvalidAddressException</code>, <code>InvalidAgeException</code>
Archivos modificados	Los 7 Value Objects del Módulo 2C: <code>FullName</code>, <code>LastName</code>, <code>Document</code>, <code>EmailAddress</code>, <code>Phone</code>, <code>Address</code>, <code>BirthDate</code>. Solo se modificaron los <code>throw</code> — el resto de la estructura permanece intacta.
Principio aplicado	Fail Fast: el dominio lanza en el momento exacto de la violación. Application captura y traduce. Nunca se valida dos veces lo mismo.
SOLID aplicado	SRP — cada excepción tiene una sola responsabilidad. OCP — nuevas excepciones no modifican las existentes. LSP — todas son sustituibles por <code>DomainException</code>. DIP — Application depende de la abstracción base.
Siguiente módulo	Módulo 4 — Integración Entity + Value Objects: construir la entidad <code>Customer</code> completa que compone los 7 VOs ya validados con sus excepciones.

CommerX · Módulo 3B completado · Continúa en → **Módulo 4: Integración Entity + Value Objects**